

Master DSA & Programming in Java

Think like a programmer. Learn the WHY before the HOW.

YOUR STARTING PROFILE

Java experience	Absolute beginner (Level 0)
DSA experience	None yet — clean slate
Prior practice	None (no LeetCode/CF)
Primary goal	Coding interviews
Study capacity	2-4 hours / day
Target date	No deadline — true mastery
Comfort now	1-3 / 10 → target 8+

This guide is your map. Keep it open beside every lesson. It contains your roadmap, diagnostic test, complexity cheat-sheet, a 15-pattern notebook, daily schedule, spaced-repetition plan, error log, and mastery checkpoints.

SECTION 0

How to Use This Guide

This is not a book you read once. It is a training companion you return to daily. It is built specifically for YOU — a complete beginner aiming at coding interviews, studying 2–4 hours a day, optimizing for real understanding rather than speed.

The one rule that matters most

Understand the WHY before the HOW.

Never jump from "here is an algorithm" straight to "here is the code." For every technique you learn, you must be able to answer: What problem does it solve? Why is the naive way too slow? What observation makes it fast? Why is it correct? If you cannot explain it in your own words, you have not learned it yet — you have only recognized it.

The learning loop — every topic follows this

Concept → Intuition → Example → Visualization → Implementation → Practice → Debugging → Optimization → Reflection

How the sections fit together

- Section 1 — Take the diagnostic test. It tells us exactly where to begin (no coding needed).
- Section 2 — Your phase-by-phase roadmap from zero to interview-ready.
- Section 3 — Java foundation: the mental models you build first.
- Section 4 — Complexity (Big-O): the language for judging solutions.
- Section 5 — The 12-step problem-solving framework you use on EVERY problem.
- Section 6 — The Pattern Notebook: 15 patterns that unlock most interview questions.
- Section 7 — Daily & weekly schedule tuned to your 2–4 hours.
- Section 8 — Spaced repetition so you don't forget what you learn.
- Section 9 — Your Error Log template (photocopy / retype it often).
- Section 10 — Mastery levels & checkpoints: how you know you're ready.
- Section 11 — Curriculum checklist + trusted resources.

A promise about how we'll work together

I will never just hand you a solution and move on. Every problem gets hints first (observation → data structure → pattern → pseudocode → code). You advance a level only when you can explain a concept back — not because you finished a lesson. We build your Error Log and Pattern Notebook together as we go.

SECTION 1

Your Diagnostic Test

Take this FIRST. It is calibrated for a beginner — it does not test Java syntax (you haven't learned it yet; testing it would be pointless). Instead it measures the raw thinking skills that predict how fast you'll pick up programming: logic, following steps precisely, spotting patterns, and breaking problems apart.

How to take it

Answer in plain English and numbers. Don't Google. Don't overthink. If you don't know, write "not sure" — that is genuinely useful data, not a failure. Write your answers in a notebook, then send them to me and I'll score you across four dimensions and confirm your exact starting point.

Part A — Logical reasoning

1. A box has 3 red balls and 2 blue balls. You pick one without looking. Are you more likely to pick red or blue? Why?
2. If every Zib is a Zab, and every Zab is a Zob, is every Zib definitely a Zob? Yes/no + one line.

Part B — Following steps precisely (this is what code IS)

3. Follow exactly and give the final value of X: start X=5; add 3; double X; subtract 6.
4. Given [4, 1, 7, 3, 9, 2], describe in your own words the step-by-step process YOU would use to find the largest number — as if the helper can only compare two numbers at a time.

Part C — Pattern recognition

5. What comes next: 2, 4, 8, 16, ___ ? What's the rule?
6. What comes next: 1, 1, 2, 3, 5, 8, ___ ? What's the rule?

Part D — Problem breakdown

7. Given a word like "racecar", how would you check whether it reads the same forwards and backwards? Describe the idea in plain words (no code).
8. A phone book has 1,000,000 names, sorted alphabetically. Checking one-by-one from the start is slow. Can you think of a FASTER way to find one name? Describe the idea.

Part E — Math comfort (quick gut-check)

9. What is 2 to the power of 10 (2 multiplied by itself 10 times)? Rough answer is fine.
10. If a task takes 1 second for 100 items, and the work doubles every time the items double, roughly how long for 400 items? Reason it out.

What each part secretly measures

A → deductive logic (the backbone of proving correctness).

B → precise step-following (a program is just precise steps) and building an algorithm in words.

C → pattern recognition (Q6 is the Fibonacci sequence — a recursion classic).

D → Q7 hints at the two-pointer pattern; Q8 hints at binary search. These are two of the most important patterns you'll learn.

E → comfort with exponential growth and doubling — the heart of Big-O intuition.

SECTION 2

Your Personalized Roadmap

Because you start at zero with strong daily time and no deadline, we optimize for depth. Depth now = speed later. Here is the honest, realistic path. Timelines assume ~2-4 focused hours/day; they flex around real understanding, never a calendar.

The big picture

Phase	Focus	Rough time
P1 Foundations	Java basics, memory model, thinking in steps	3-4 weeks
P2 Java for DSA	OOP + the collections you'll actually use	2-3 weeks
P3 Core DSA I	Arrays, strings, hashing, two-pointers, sliding window	3-4 weeks
P4 Recursion	Recursion, backtracking, sorting, binary search	3-4 weeks
P5 Core DSA II	Linked lists, stacks, queues, trees, heaps	4-5 weeks
P6 Advanced	Graphs, dynamic programming, greedy, tries	5-7 weeks
P7 Interview mode	Patterns, mock interviews, timed sets, company-style	ongoing

Honest expectation from a true zero start with serious daily study: solid Java + core DSA in roughly 2-3 months; genuinely interview-ready in roughly 4-6 months. Your no-deadline choice means we NEVER skip a shaky concept to hit a date.

Phase details & exit criteria

Phase 1 — Foundations (you are here)

- Learn: JVM/JDK/JRE, source→bytecode, variables, primitive vs reference types, operators, I/O, conditionals, loops, methods, arrays, strings.
- Build the memory model: stack vs heap, references vs objects.
- ✓ Exit test: read a small program and predict its output by tracing it line-by-line.

Phase 2 — Java for DSA

- OOP: classes, objects, constructors, this, encapsulation, inheritance, polymorphism, interfaces, abstract classes.
- Collections you'll use constantly: String/StringBuilder, ArrayList, HashMap, HashSet, ArrayDeque, PriorityQueue, TreeMap, and Comparator/Comparable.
- ✓ Exit test: pick the right collection for a given task and justify why.

Phase 3 — Core DSA I

- Arrays & strings mastery; prefix sums; frequency maps; two pointers; sliding window; Kadane's algorithm.
- Hashing: complement lookup, frequency counting, prefix-sum + hashmap, duplicate detection, grouping.
- ✓ Exit test: solve fresh easy/easy-medium array & string problems and analyze their complexity.

Phase 4 — Recursion, sorting, binary search

- Recursion from first principles (base case, recursive case, call stack, recursion tree).
- Backtracking (Choose → Explore → Undo): subsets, permutations, combinations.
- Sorting internals + binary search, including 'binary search on the answer'.

- ✓ Exit test: trace a recursion tree by hand and identify a binary-searchable answer space.

Phase 5 — Core DSA II

- Linked lists (pointer manipulation, fast/slow pointers), stacks (monotonic stack), queues/deques, trees (traversals, BST, LCA), heaps/priority queues (Top-K).
- ✓ Exit test: implement a tree traversal and a Top-K solution from memory.

Phase 6 — Advanced

- Graphs: BFS/DFS, topological sort, Dijkstra, Union-Find, MST. Dynamic programming as a reasoning skill (state → choice → recurrence → base → memo → tabulate). Greedy. Tries.
- ✓ Exit test: for a DP problem, clearly state what `dp[i]` means before writing any code.

Phase 7 — Interview mode (ongoing)

- Pattern recognition drills, timed sets, mock interviews with scoring, company-style problem sets by topic, and communication practice.

SECTION 3

Java Foundation — First Mental Models

You start here. The goal of Phase 1 is not to memorize syntax — it is to build correct mental models so nothing feels like magic later.

How Java runs

You write source code (.java). The compiler (javac) turns it into bytecode (.class) — a portable instruction set. The JVM (Java Virtual Machine) executes that bytecode on any machine. That's the famous "write once, run anywhere."

JDK = the full kit to build Java (includes the compiler). JRE = just enough to run Java. JVM = the engine inside the JRE that runs bytecode.

THE PIPELINE

```
Your code (.java) → javac → bytecode (.class) → JVM → runs
```

Your first program

HELLO.JAVA

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello, world!");  
    }  
}
```

Every line has a reason — we'll unpack public, class, static, void, main, and String[] args one at a time in the lessons. For now: main is where the program starts, and println prints a line.

The memory model (this prevents SO many future bugs)

TWO VERY DIFFERENT THINGS

```
int x = 10;  
int[] arr = {1, 2, 3};
```

x is a primitive. The value 10 lives directly in the variable, on the stack. Copy x and you copy the value.

arr is a reference. The variable holds an address that points to an array object living on the heap. Copy arr and you copy the address — both names now point to the SAME array. Change it through one, and the other sees the change.

PRIMITIVE VALUE VS REFERENCE



Common beginner misconception

"Assigning one array to another makes a copy." It does NOT. `int[] b = arr;` makes `b` point at the same array as `arr`. To truly copy, you must clone/copy the elements (e.g. `Arrays.copyOf`). This single misunderstanding causes a huge share of early bugs — remember the diagram.

Phase-1 checklist (tick when you can DO it, not just read it)

- ☐ Print output and read a value
- ☐ Declare primitives & understand each type
- ☐ Use if / else and switch
- ☐ Write for, while, and do-while loops
- ☐ Write a method that takes inputs and returns a value
- ☐ Create and loop over an array
- ☐ Build a String and modify with `StringBuilder`
- ☐ Trace a 15-line program's output by hand

SECTION 4

Complexity (Big-O) Cheat Sheet

Big-O describes how the work grows as input size n grows. It is the language you use in interviews to judge whether a solution is good. You will learn to CALCULATE it, not memorize it — but keep this table close.

Big-O	Name	$n=10$	$n=1,000$	Feels like
$O(1)$	Constant	1	1	Instant — array index, hashmap get
$O(\log n)$	Logarithmic	~ 3	~ 10	Very fast — binary search
$O(n)$	Linear	10	1,000	One pass over the data
$O(n \log n)$	Linearithmic	~ 33	$\sim 10,000$	Good sorting
$O(n^2)$	Quadratic	100	1,000,000	Nested loops — slow at scale
$O(2^n)$	Exponential	1,024	astronomical	Brute-force subsets
$O(n!)$	Factorial	3.6M	astronomical	All permutations

How to read a loop

PATTERN → COMPLEXITY

```

for (int i = 0; i < n; i++) { ... }           //  $O(n)$ : runs  $n$  times
for (i..n) for (j..n) { ... }               //  $O(n^2)$ :  $n \times n$ 
for (i = n; i > 1; i = i / 2) { ... }        //  $O(\log n)$ : halves each step
for (i..n) for (j..n) for (k..n) { ... }    //  $O(n^3)$ 

```

Rules of thumb: sequential loops ADD ($n + n = O(n)$); nested loops MULTIPLY ($n \times n = O(n^2)$); dividing the range by 2 each step is $O(\log n)$; drop constants and lower-order terms ($O(2n+5) \rightarrow O(n)$).

Space complexity

Same idea, but for extra memory you allocate. An extra array of size n is $O(n)$ space; a few variables is $O(1)$. Recursion also uses stack space proportional to its depth — a recursion n deep is $O(n)$ space even with no arrays.

The interview reflex

For every solution, state BOTH time and space out loud, then ask: "Can I do better on either?" The classic trade-off is spending $O(n)$ memory (e.g. a HashMap) to cut time from $O(n^2)$ to $O(n)$. Recognizing that trade is half of interview problem-solving.

Practice — calculate these yourself

- Two separate (not nested) loops each from 0 to n . Total?
- A loop 0.. n , and inside it a binary search over n items. Total?
- Recursion that makes 2 calls each level, depth n . Time? Space?

(Answers we'll check together — reason first, then verify.)

SECTION 5

The Problem-Solving Framework

Use these 12 steps on EVERY problem until they become automatic. Most beginners fail interviews not because they can't code, but because they start coding before thinking. This framework fixes that.

- | | |
|-------------------------|--|
| 1 Understand | Restate the problem in your own words. What exactly is asked? What's the input and output? |
| 2 Constraints | How big is n? Are values sorted? Any negatives? Constraints hint at the target complexity. |
| 3 Brute force | What is the simplest correct solution, even if slow? Always have one. |
| 4 Bottleneck | What makes brute force too slow? Find the expensive part. |
| 5 Observation | What structure or pattern can you exploit? (sorted? repeats? monotonic?) |
| 6 Data structure | Which structure removes the bottleneck? (HashMap? Heap? Stack?) |
| 7 Algorithm | Which pattern/algorithm fits? (two pointers, BFS, DP, ...) |
| 8 Proof | Why is it correct? What invariant always holds? |
| 9 Complexity | State time AND space. Is it good enough for the constraints? |
| 10 Edge cases | Empty input, one element, duplicates, negatives, overflow, all-same. |
| 11 Implement | Now — and only now — write clean Java. |
| 12 Test | Dry-run on a small example and the edge cases before you claim done. |

The DP litmus test

For any dynamic-programming problem, before you write a single line you must answer: "What does dp[i] MEAN?" If you can't state that in one clear sentence, you're not ready to code it yet. This one habit prevents most DP confusion.

SECTION 6

The Pattern Notebook (15 Patterns)

Most interview problems are variations of a small set of patterns. Learn to RECOGNIZE them and you'll rarely be truly stuck. For each: when to reach for it, the core idea, and its typical complexity. Templates and full examples come during the lessons — this is your quick-recall index.

1. Two Pointers

Recognize when Sorted array/string, or comparing from both ends; find a pair/triplet, remove duplicates in place, reverse.

Core idea: Two indices move toward each other or in the same direction to avoid a nested loop.

Complexity: $O(n)$ time, $O(1)$ space

2. Sliding Window

Recognize when: "Longest/shortest/contains" over a contiguous subarray or substring.

Core idea: Grow the window on the right; shrink from the left when a condition breaks. Reuse work instead of recomputing.

Complexity: $O(n)$ time, $O(1)$ – $O(k)$ space

3. Prefix Sum

Recognize when Many range-sum queries, or count subarrays summing to k .

Core idea: Precompute running totals so any range sum is one subtraction. Pair with a hashmap for subarray counts.

Complexity: $O(n)$ build, $O(1)$ query

4. Hash Map Lookup

Recognize when: "Have I seen X?", complements (two-sum), frequency counts, grouping.

Core idea: Trade $O(n)$ memory to turn an $O(n^2)$ search into $O(n)$ by remembering what you've seen.

Complexity: $O(n)$ time, $O(n)$ space

5. Fast & Slow Pointers

Recognize when Linked-list cycle, find middle, n th-from-end.

Core idea: Two pointers at different speeds; they meet, or one reaches the end first.

Complexity: $O(n)$ time, $O(1)$ space

6. Binary Search

Recognize when Sorted data, OR a monotonic answer space ("smallest x that works").

Core idea: Halve the search space each step. Ask: is the yes/no answer monotonic in x ?

Complexity: $O(\log n)$ time

7. Monotonic Stack

Recognize when Next greater/smaller element; histogram; spans.

Core idea: Keep a stack in increasing or decreasing order; pop when the order would break.

Complexity: $O(n)$ time, $O(n)$ space

8. Heap / Top-K

Recognize when: "K largest/smallest", median of a stream, merge K lists.

Core idea: A size- K heap keeps the best K seen so far; push/pop in $O(\log k)$.

Complexity: $O(n \log k)$ time

9. BFS

Recognize when Shortest path in an UNweighted graph; level-by-level traversal.

Core idea: A queue explores nearest-first, so the first time you reach a node is via a shortest path.

Complexity: $O(V + E)$

10. DFS

Recognize when Explore all paths; connectivity; tree/graph recursion.

Core idea: Go as deep as possible, then backtrack. Natural fit for recursion.

Complexity: $O(V + E)$

11. Backtracking

Recognize when Generate all subsets/permutations/combinations; constraint puzzles (N-Queens).

Core idea: Choose → Explore → Undo. Build a partial solution, recurse, then remove the choice.

Complexity: often $O(2^n)$ or $O(n!)$

12. Greedy

Recognize when Local best choice leads to global best (intervals, scheduling).

Core idea: Sort by the right key, then take the locally optimal option each step. Must prove it's safe.

Complexity: usually $O(n \log n)$

13. Dynamic Programming

Recognize when Overlapping subproblems + optimal substructure; count/optimize over choices.

Core idea: Define $dp[state]$; write a recurrence from choices; memoize or tabulate.

Complexity: state count $\times$ work

14. Union-Find (DSU)

Recognize when Connected components, cycle detection in undirected graphs, grouping.

Core idea: Track set membership with find + union; path compression makes it near $O(1)$.

Complexity: $\sim O(\alpha(n))$ per op

15. Topological Sort

Recognize when Ordering with dependencies (course schedule, build order) in a DAG.

Core idea: Repeatedly take nodes with no remaining prerequisites (Kahn's / DFS post-order).

Complexity: $O(V + E)$

SECTION 7

Your Daily & Weekly Schedule

Tuned to your 2–4 hours/day. Consistency beats intensity — a steady 2 hours daily crushes an occasional 8-hour cram. Here are two ready-made daily plans; pick by how much time you have that day.

Daily plan — the ~2 hour day

Time	Activity	Why
20 min	Concept learning	Learn one new idea + its intuition
25 min	Implement from scratch	Type it yourself — no copy/paste
50 min	Problem solving	1–2 problems using the 12-step framework
15 min	Debug / review	Fix what broke; log any mistake
10 min	Complexity + reflection	State Big-O; write 1 line on what clicked

Daily plan — the ~3–4 hour day

- 30 min concept + intuition · 40 min implement-from-scratch + a memory rebuild of an old topic
- 90 min problem solving (2–3 problems, mixing a new pattern with a spaced-repetition review)
- 30 min debugging + Error Log entries · 20 min complexity analysis + reflection

Weekly rhythm

Day	Emphasis
Mon–Thu	New concepts + fresh problems (the core learning days)
Fri	Pattern day: 3–4 problems that all use one pattern back-to-back
Sat	Timed challenge (see Contest training) + one mock-interview question
Sun	Review: Error Log, Pattern Notebook, and any topic you keep forgetting

End-of-week self-test

Every Sunday, answer in writing: What concepts did I learn? What problems did I solve? Which patterns are getting solid? What are my top 2 weak areas? Then I'll give you a short mixed quiz. If you fail a concept, we revisit it BEFORE moving on — never advance past a shaky idea.

SECTION 8

Spaced Repetition — Don't Forget What You Learn

Solving something once does NOT mean you know it. Memory fades on a predictable curve; we beat it by revisiting each concept at growing intervals. When you learn topic X, schedule these touch-points:

When	What to do with the concept
Day 1	Learn it fully — concept, intuition, one worked example.
Day 2	Recall from memory: explain it out loud without notes.
Day 4	Practice: solve one problem using it.
Day 7	Mixed problem: solve it among other patterns (no hint that it's this one).
Day 14	Timed problem: solve under a clock.
Day 30	Review: re-derive the template from scratch.

The forgetting tracker

Keep a simple list titled "Concepts I keep forgetting." Every time you have to look something up that you'd already 'learned', add a tally. Anything with 3+ tallies gets a dedicated review session. Over time this list becomes the highest-value study plan you own.

SECTION 9

Your Error Log

Every serious mistake is a gift — it shows exactly what to fix. For each significant error, fill one entry. Over time this log personalizes your entire training toward your real weak spots. Copy this template into a notebook or a file and reuse it constantly.

Template

Problem	What was the problem? (name / link / one-line description)
Mistake	What did I actually do wrong?
Root cause	WHY did I make it? (misread? wrong data structure? off-by-one? bad assumption?)
Correct concept	What should I have recognized or recalled?
Prevention	A concrete rule to avoid it next time.
Similar problem	One problem to redo that tests the same thing.

Worked example (so you see the depth expected)

Problem	Two Sum — return indices of two numbers adding to target.
Mistake	Used two nested loops → $O(n^2)$; also returned values instead of indices.
Root cause	Jumped to brute force and didn't ask 'have I seen the complement?'; misread the output spec.
Correct concept	Hash Map complement lookup: store value→index; check target−x in one pass → $O(n)$.
Prevention	Re-read the required OUTPUT before coding; ask 'can a hashmap remember what I need?'
Similar problem	Two Sum II (sorted input) — solve with two pointers instead, and compare.

Common bug categories to tag your entries with

compile error	runtime exception	logic error	off-by-one	null reference
infinite loop	stack overflow	wrong state	wrong complexity	misread problem

SECTION 10

Mastery Levels & Checkpoints

You advance by DEMONSTRATING skill, never by finishing lessons. Here is the full ladder and what each rung requires.

Level	Name	You can...
0	Absolute beginner	(you start here) — think in steps, follow logic
1	Java fundamentals	Write & trace loops, methods, arrays, strings
2	Basic problem solving	Turn a simple problem into working code alone
3	Core DSA	Use the main data structures correctly
4	Pattern recognition	Spot which pattern a new problem needs
5	Medium problems	Solve fresh mediums with light hints
6	Advanced DSA	Graphs & DP with confidence
7	Hard problems	Combine patterns; solve hard problems
8	Interview ready	Solve + explain + analyze under time & pressure
9	Contest capable	Solve unfamiliar problems fast
10	DSA mastery	Look at anything new and reason it out end-to-end

Example checkpoint — Arrays

Before leaving arrays, you must be able to, without notes:

- ✓ Apply two pointers to a sorted-array problem
- ✓ Apply a sliding window to a subarray problem
- ✓ Compute and use a prefix-sum array
- ✓ Recognize an interval problem
- ✓ State the time & space complexity of your solution

Fail any one? We revisit that specific concept before moving forward. That's the whole point of checkpoints.

The ultimate goal (read this often)

What DSA mastery really means

Not "how many problems did I finish." The real target: can I look at a NEW problem, understand it, identify useful patterns, develop an approach, argue why it's reasonable, implement it cleanly in Java, debug it, analyze its complexity, and explain my solution clearly? Everything in this guide is built to get you there.

SECTION 11

Curriculum Checklist & Resources

The full topic map. Tick each box only when you can DO it and explain it — not merely when you've read about it.

Java core

- ☐ Execution model & memory model
- ☐ Variables, types, operators, I/O
- ☐ Conditionals & loops
- ☐ Methods & recursion basics
- ☐ Arrays & Strings
- ☐ OOP: classes, objects, inheritance, interfaces, polymorphism
- ☐ Exceptions & generics

Java for DSA

- ☐ String / StringBuilder
- ☐ ArrayList / LinkedList
- ☐ HashMap / HashSet / TreeMap / TreeSet
- ☐ ArrayDeque (stack & queue)
- ☐ PriorityQueue
- ☐ Comparator / Comparable
- ☐ Arrays, Collections, Math utilities

Data structures

- ☐ Arrays & strings
- ☐ Linked lists
- ☐ Stacks & queues / dequeues
- ☐ Hashing
- ☐ Trees & BSTs
- ☐ Heaps / priority queues
- ☐ Graphs
- ☐ Tries
- ☐ Union-Find, segment/Fenwick trees (advanced)

Algorithms & patterns

- ☐ Two pointers, sliding window, prefix sum
- ☐ Binary search (incl. on the answer)
- ☐ Sorting (merge, quick, heap)
- ☐ Recursion & backtracking
- ☐ BFS / DFS & graph algorithms
- ☐ Greedy
- ☐ Dynamic programming (1D, 2D, knapsack, LIS, LCS)

Trusted resources (free)

- Practice: LeetCode (start with Easy, then the pattern-based lists).
- Theory & references: GeeksforGeeks, and the official Java documentation.

- Visual intuition: VisuAlgo (algorithm animations) and USFCA Data-Structure Visualizations.
- Complexity: Big-O cheat sheet (bigocheatsheet.com) — but always try to derive it yourself first.

You don't need paid courses to reach interview-ready. You need this guide, consistent daily reps, and the discipline to always ask WHY.

Your very next step

Do this today

1) Take the diagnostic in Section 1 and send me your answers. 2) Set up your Error Log and 'Concepts I keep forgetting' list. 3) Then we begin Phase 1, Lesson 1: how Java runs + your first program — taught the right way, concept and intuition before any code. You've got real daily time and the right mindset. Let's build this properly.